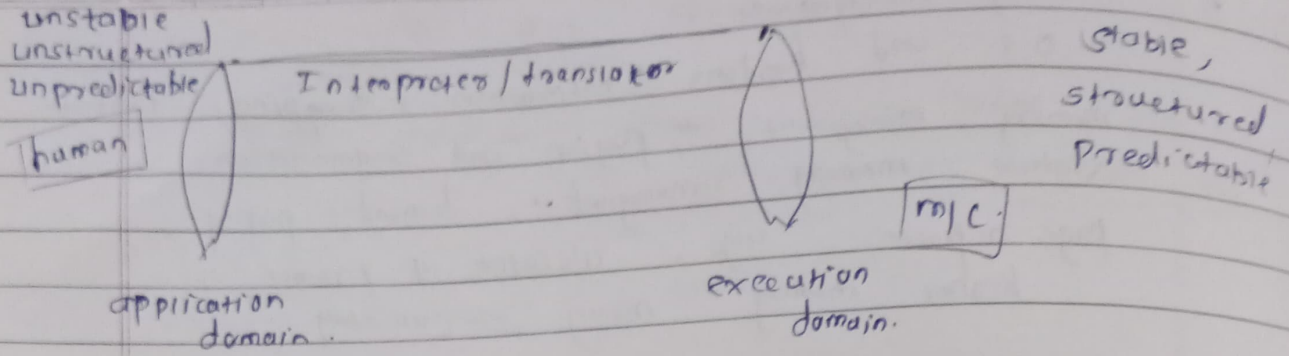


## Software



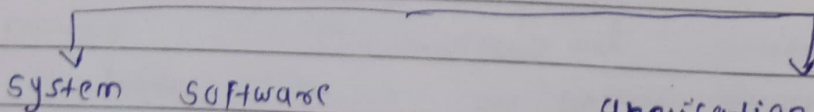
## Computer

- (i) pulley and mech.
- 1st toggle switch
- transistor
- SSI
- LSI
- VLSI

def<sup>n</sup>: To bridge gap between application domain & symmetric gap b/w application domain (human) and execution domain (m/c) is bridged by a logical entity termed as a software.

software is group of programs that help in bridging the symmetric gap b/w application domain and execution domain.

## Types of Software



system software

application software

special system software

software tools

operating system

eg. banking software,   
 ~~Public~~ Railway Reservation

eg. editor, debugger etc.

(work is only to translate things)  
ex: compiler, assembler

PAGE No.   
 DATE / /

OS. is a System software that takes care of operation, execution, and maintenance of valuable resources (hardware)

### → System Software

System software are the programs which aid in effective execution of general user computational requirements of computer system.

S.S. is the collection of programs which are needed in creation, preparation, execution of user program.

S.S. are the programs which utilize resources of system in optimized way and help the user to translate their programs and execute it.

### × Translators

List:

Compiler,

Interpreter,

Assembler,

Linker

Loader

Cross compiler

Simulator

Debugger

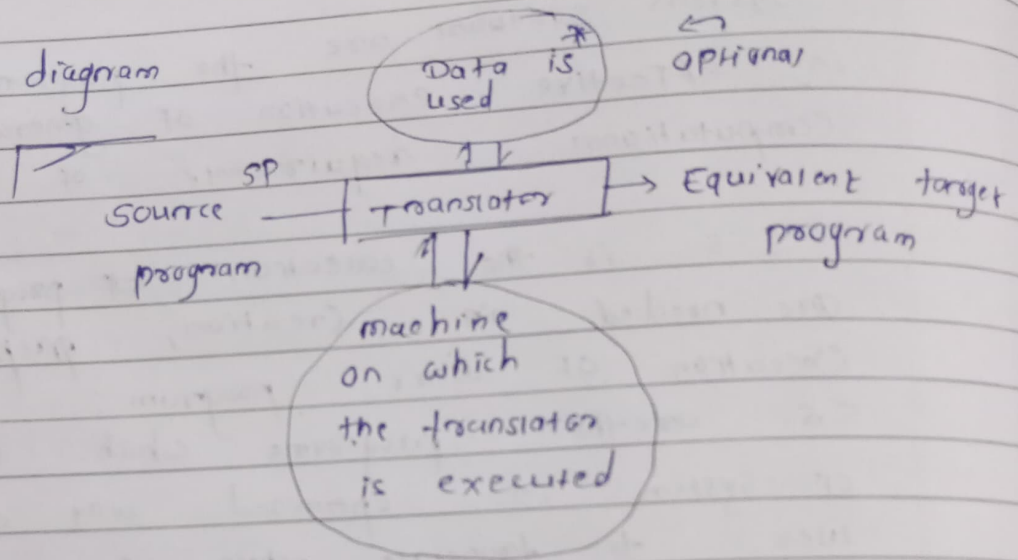
Preprocessors,

Macroprocessors

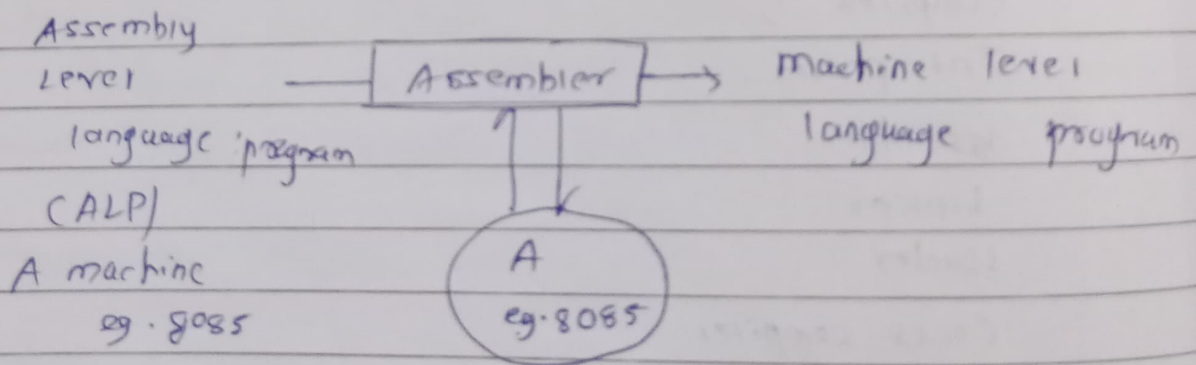
Discompiler or disassembler

Any translator denoted by  $S \xrightarrow{T} T_n$   
 $T \rightarrow$  translator  
 $T \rightarrow$  target language  
 $S \rightarrow$  source language  
 $n \rightarrow$  machine on which translator is executed.

block diagram



## ① Assembler



files

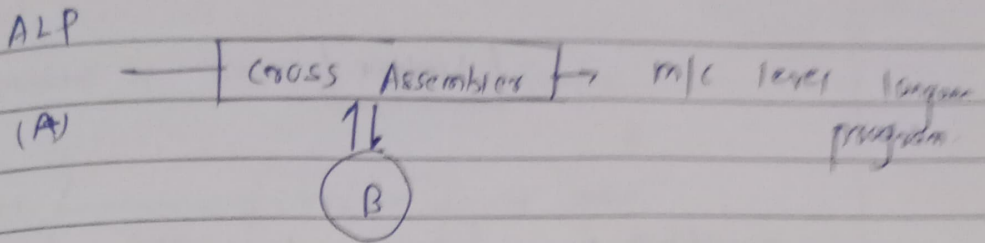
.Lst

.Obj

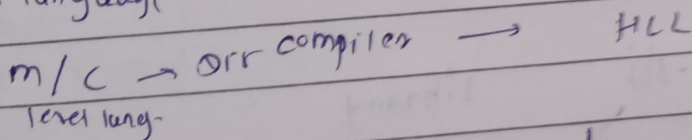
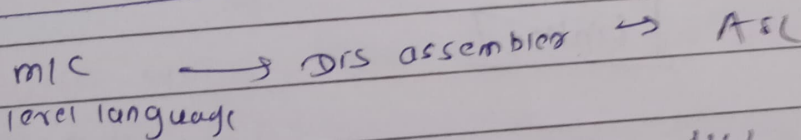
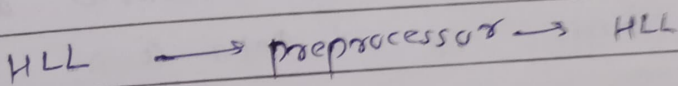
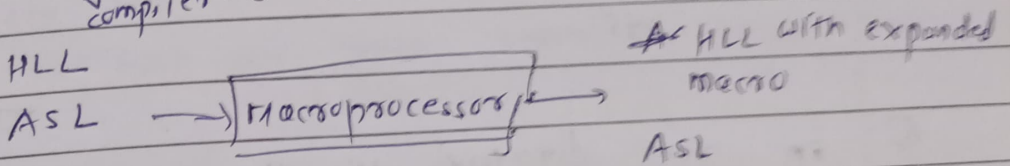
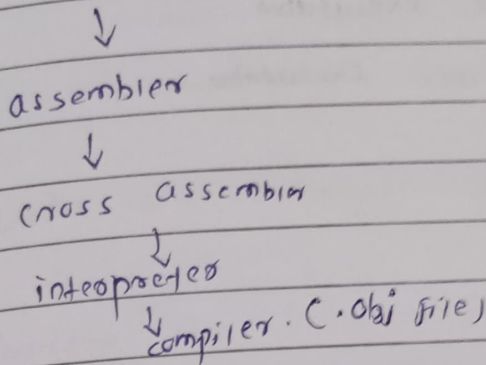
.Lst: Information regarding address enter is there in .Lst file.

.Obj: Final m/c code of source program along with address information

|    |      |     |
|----|------|-----|
| HW | DATE | / / |
|----|------|-----|



Translators:-



Is it feasible to design disassembler or  
discompiler

## Linker

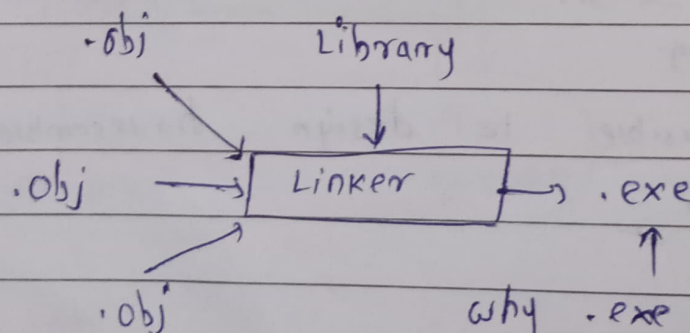
in addition to binary code of source program there is some address info in .obj file which is used by linker.

.obj file is not executable.  
extensions which are executable

- .exe
- .out
- .bak
- .com
- .sys

Q In root directory if there are files which file will be executed and on what basis if i enter a

## Linker



why .exe file created by linker is still not executable.

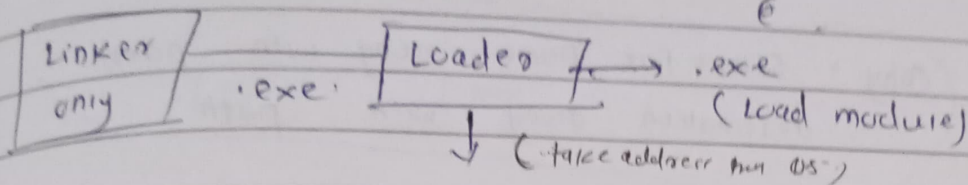
→ Starting address should be allocated.

but it is not assigned by linker

hence →

Loader

File created by linker is ip to Loader.



Call O.S

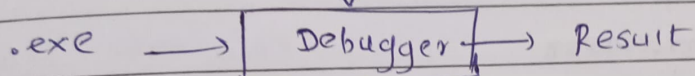
Memory management module.

Loader will reallocate the program file.

Debugger

To find out logical errors in program.

↓ data.



O.S (also called as resource manager.)

takes care of valuable resources like.

eg. CPU, memory, I/O devices,

file system, protection and security.

→ main task O.S

1) It should provide interface. betn user and Hard ware (HW)

2) Resource allocation.

3) manage resources efficiently

→ Goals of O.S (depends on m/c type)

Home made m/c

convenience ↑  
efficiency ↓

Super computer

↓  
↑

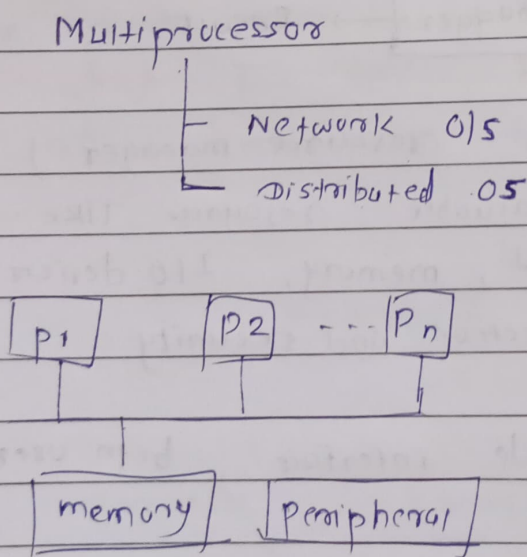
- i) convenience
- ii) efficiency

Copy : source file name along with path  
 destination along with path

\* Types of O.S :

- ① Batch O.S. ✓
- ② multiprogramming ✓
- ③ multitasking
- ④ time sharing
- ⑤ multiprocessing (distributed) ✓
- ⑥ Real time ✓

4/8/24

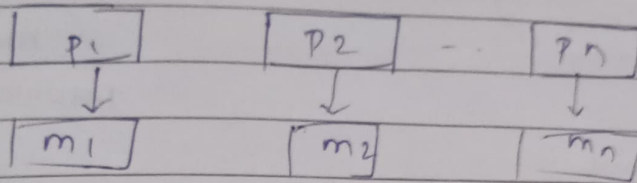


loose couple  
 microprocessor  
 OS

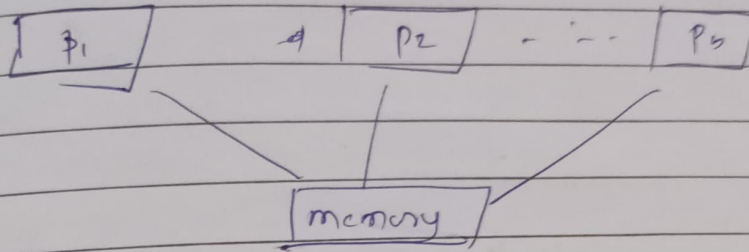
tightly couple  
 microprocessor OS

loosely couple :

→ fast

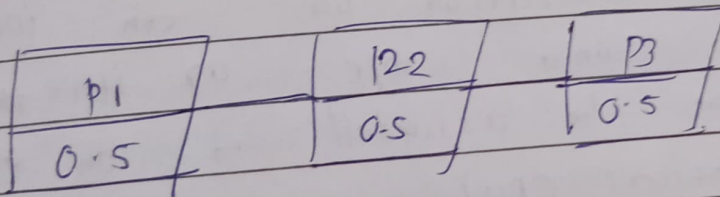


tightly couple :

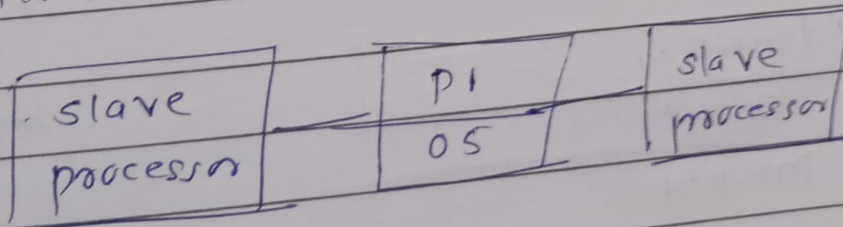


if ~~all~~ 1 processor fail , next take the operation in tightly.

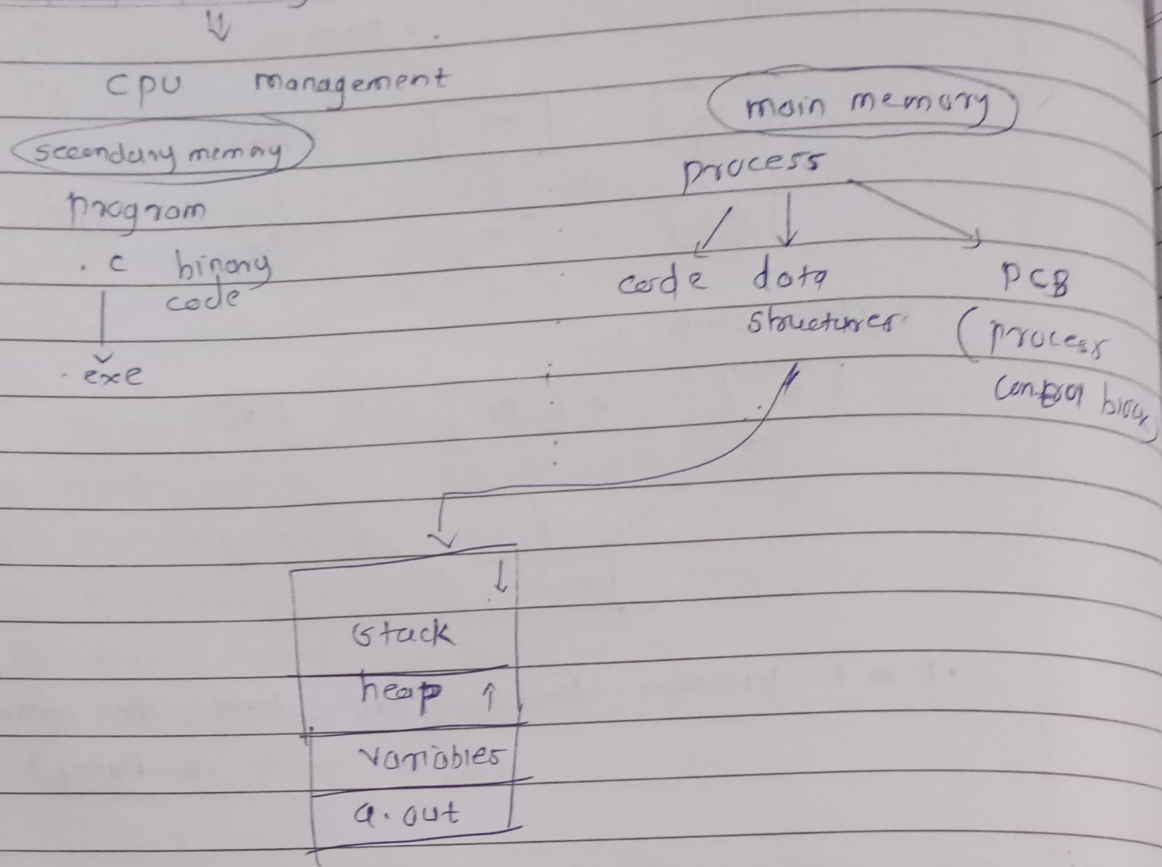
- ① separate supervisor  
each processor have separate copy of OS.



- ② master slave



# Process management



Process is fundamental task of O.S.

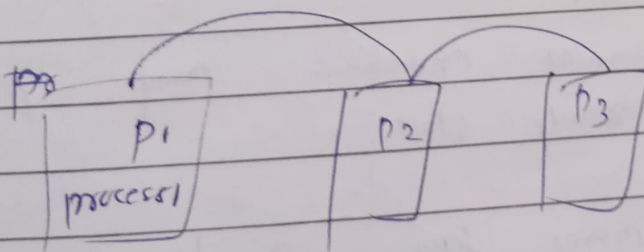
It is a program execution or an instance of a program running on a m/c, or it is the entity that can be assigned to an executed on the processor (CPU)

## \* PCB

|                         |
|-------------------------|
| Identifier              |
| state                   |
| priority                |
| program counter         |
| memory pointers         |
| context data            |
| IO status & information |
| Accounting Info         |

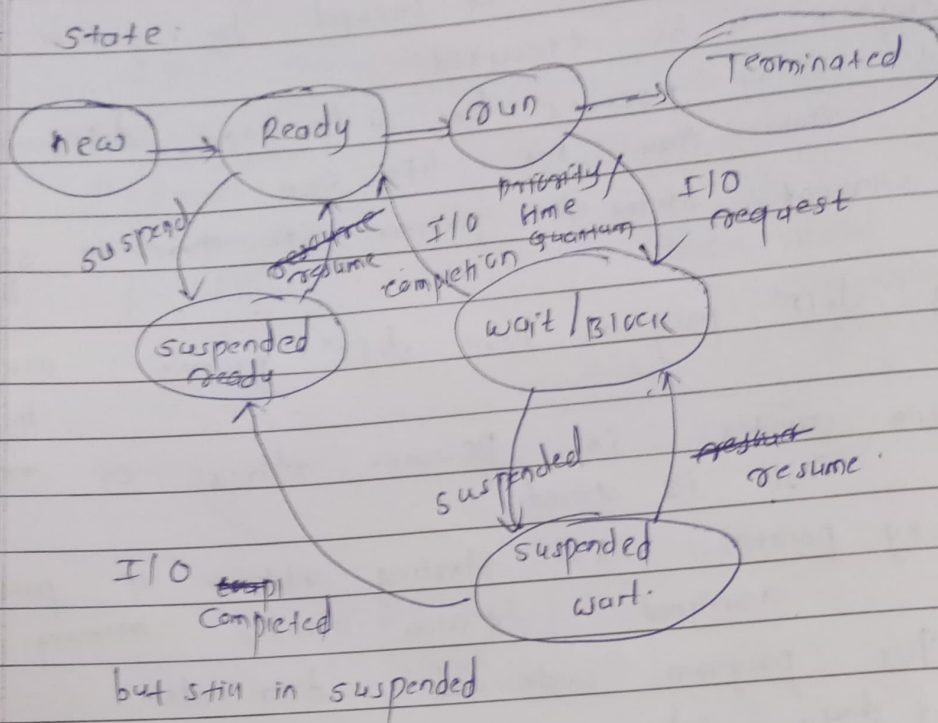
Advanced : 0.5

- Identifies its unique id created by process for program to be executed.
- state: every process has to be pass different states thru his life cycle  
current status of process is there in state blocks
- priority: diff<sup>n</sup> programs have diff<sup>n</sup> priority. assigned by O.S.
- program counter: in PC register address of next inst. is stored.
- memory pointer: It is starting address of process assigned. It also includes memory ptr. for program code and data code.
- context data: when program is executing if user some registers that contains an info is a context data.
- IO status info: whenever a process is requesting any io and if it is assigned to that process then its status is mentioned in io status block.
- Acc. info: this may include amount of processor time clock time used time limits etc.



Linked list is used for program context block

\* state



→ New: A process is said to be in new state when program present in secondary memory and initiated for execution.

→ Ready state: A process moves from new state to ready state after it is loaded into main memory and now it is ready for execution.

In ready state the process waits for execution by the processor.

In multi programming environment many processes are in ready state.

→ Run: process moves from ready to run state after it is assigned CPU for execution.

- terminated state: A process moves from run to term. after its execution completed, then PCB associated with it is also deleted.

Block/wait state: A process moves from run to block state by 3 reasons.

- (1) it request io
- (2) high priority process is assigned for exec.
- (3) time quantum finished in time sharing o.s.

suspended wait state: ready

A process moves from ready to sus. ready if process of high prio. has to be executed but main memory is full.

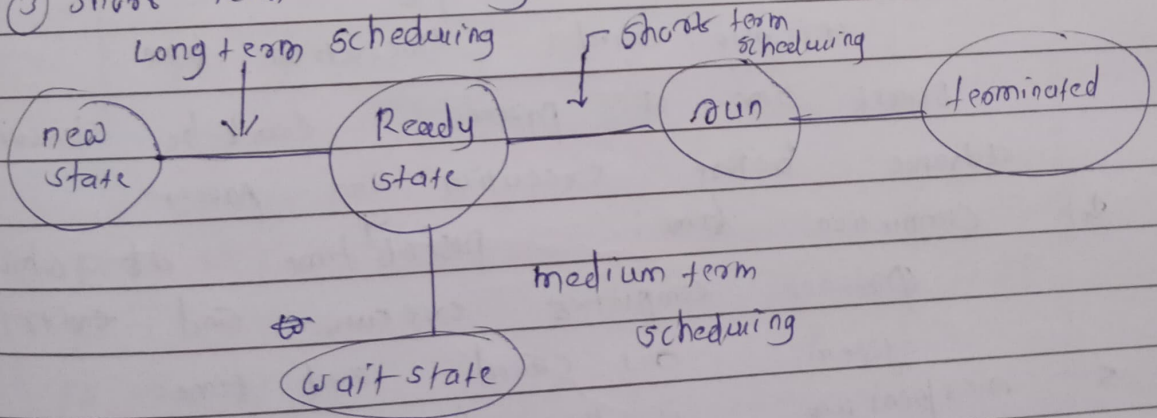
suspended wait.

A process moves from wait state to susp. wait if process with high prio has to be executed with some <sup>io</sup> devices and main memory is full.

CPU Scheduling:

Scheduling policies

- (1) long term scheduling. → All situations
- (2) medium term
- (3) short term scheduling (memory swapping)



## short term scheduling.

Timing CPU scheduling.

~~primitive~~ preemptive  
scheduling

Non ~~primitive~~ preemptive.  
Scheduling

### Timings

- i) Arrival time.
- ii) Waiting time.
- iii) ~~to~~ Burst time
- iv) Completion time
- v) response time
- vi) Turn around time.

1) Arrival time :- when a process enters into ready q. that time is called arrival time.

2) Waiting time: it is amount of time spent by process waiting in ready q.

3) Burst time :- amount of time required by process for execution on cpu.  
it is also called execution time

Burst time of process can't be known in advance before execution in process.

4) Completion time: <sup>of</sup> time at which process completes execution and exits from system. also called exit time.

5) response time: is the time when process gets the cpu time for the first time after entering into ready q.

waiting time

Burst time

|          |     |
|----------|-----|
| Page No. |     |
| DATE     | / / |

turn around time: Is the total amount of time spent by a process in the system.

when process is running then it will not exit execution  $\rightarrow$  non preemptive process

FCFS  $\rightarrow$  First come first serve.

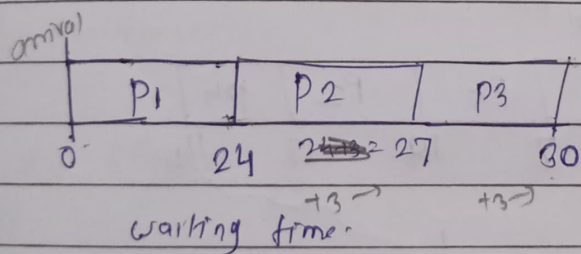
when high prio process come other process has to turn off  $\rightarrow$  preemptive.

SJF: shortest job first

Ex: consider following processes with their cpu burst time. then calculate the avg. waiting time.

| processes | Burst time | FCFS |
|-----------|------------|------|
| P1        | 24         |      |
| P2        | 3          |      |
| P3        | 3          |      |

Gantt chart



P1  $\rightarrow$  0

P2  $\rightarrow$  24

P3  $\rightarrow$  27

$$\text{avg} = \frac{0 + 24 + 27}{3} = 17$$

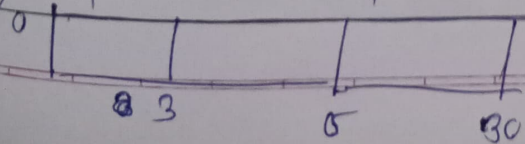
|    |    |
|----|----|
| P1 | 3  |
| P2 | 3  |
| P3 | 24 |

P1  $\rightarrow$  0

P2  $\rightarrow$  3

P3  $\rightarrow$  5

convex effect



$$\text{avg} = \frac{0 + 3 + 5}{3} = 3$$

consider set of 5 process whose arrival and burst time is as below. calculate avg. waiting time

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| P1      | 3            | 4          |
| P2      | 5            | 3          |
| P3      | 0            | 2          |
| P4      | 5            | 1          |
| P5      | 4            | 3          |

→

FCFS

| Process | A-time | Burst time |
|---------|--------|------------|
| P3      | 0      | 2          |
| P1      | 3      | 4          |
| P5      | 4      | 3          |
| P2      | 5      | 3          |
| P4      | 5      | 1          |

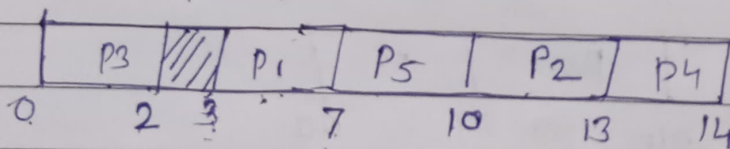
Assume:  
P3 comes first

G.

CPU

~~total~~ total time

P5 has to wait because P1 was executing.



$$\frac{0 + 3 + 7 + 10 + 13}{5} = \frac{33}{5} = 6.6$$

Calculate turn around time.

turn around time

$$p_3 \rightarrow 2 - 0 = 2$$

$$p_1 \rightarrow 7 - 3 = 4$$

$$p_5 \rightarrow 10 - 4 = 6$$

$$p_2 \rightarrow 13 - 5 = 8$$

$$p_4 \rightarrow 14 - 5 = 9$$

~~avg. time~~

Preemptive

SJF  $\rightarrow$  Shortest job first.

policy

ex:

(implement preemptive by using non preemptive)

① LJF

longest

② Priority

③ round robin (time sharing)

④ shortest remaining time

⑤ longest - 1 -

⑥ Highest response ratio next

⑦ multiple queue scheduling.

⑧ multilevel - 1 -

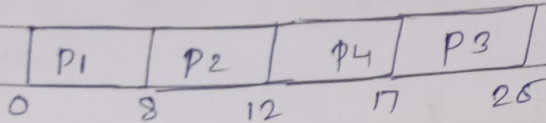
Q. consider situation of four process

|       |       | Arrival time | Burst time |
|-------|-------|--------------|------------|
| $p_1$ |       |              |            |
| $p_2$ | $p_1$ | 0            | 8          |
| $p_3$ | $p_2$ | 1            | 4          |
| $p_4$ | $p_3$ | 2            | 9          |
|       | $p_4$ | 3            | 5          |

find avg. waiting time by policy SJF  
with preemption and without preemption.

SJF

Non-preemptive.



Waiting

time

$$P_1 \rightarrow 0$$

$$P_2 \rightarrow 8 - 1 = 7$$

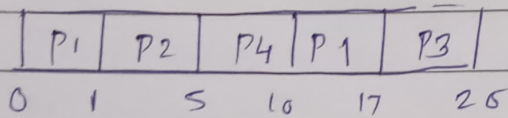
$$P_3 \rightarrow 12 - 3 = 9$$

$$P_4 \rightarrow 17 - 2 = 15$$

$$\text{avg} = \frac{0 + 7 + 9 + 15}{4}$$

$$= \frac{31}{4} = 7.75$$

Preemptive.



Waiting time

$$P_1 \rightarrow 10 - 1 = 9$$

$$P_2 \rightarrow 1 - 1 = 0$$

$$P_3 \rightarrow 17 - 2 = 15$$

$$P_4 \rightarrow 5 - 3 = 2$$

$$\text{avg} = \frac{9 + 0 + 15 + 2}{4} = \frac{26}{4} = 6.5$$

Q

|                | Arrival | Burst Time |
|----------------|---------|------------|
| P <sub>1</sub> | 0       | 7          |
| P <sub>2</sub> | 2       | 4          |
| P <sub>3</sub> | 4       | 1          |
| P <sub>4</sub> | 5       | 4          |

waiting time

ans avg Non prece 15

preemptive: 3

Policy time sharing  
↓

Round Robin policy

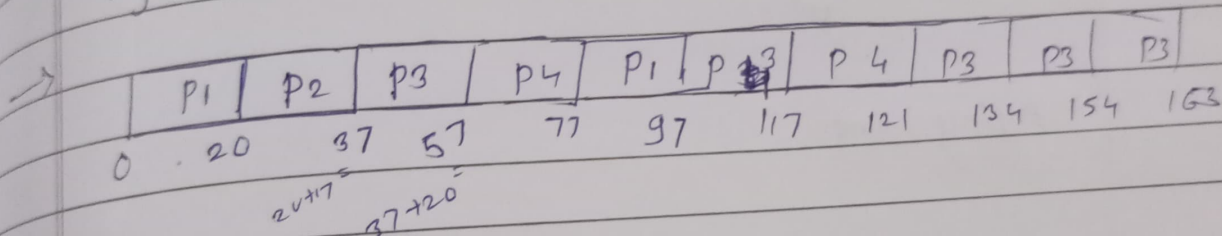
FCFS

Burst time

time unit = 20

|    |    |     |                          |
|----|----|-----|--------------------------|
| P1 | 53 |     |                          |
| P2 | 17 | Ans | $53 - 20 = 33 - 20 = 13$ |
| P3 | 68 |     | $17 - 17 = 0$            |
| P4 | 24 |     | $68 - 20 = 48 - 20 = 28$ |
|    |    |     | $24 - 20 = 4 - 4 = 0$    |

Avg waiting time 29



$$0 + (77 - 20) + (121 - 97) = -$$

consider

Burst time

Quantum time = 5 unit for each process

P1 30

Avg waiting time ?

P2 6

Ans: 15 unit

P3 8

## Priority policy.

|    | Turn time. | Priority |
|----|------------|----------|
| P1 | 10         | 3        |
| P2 | 1          | 1        |
| P3 | 2          | 3        |
| P4 | 1          | 4        |
| P5 | 5          | 2        |

Sometimes arrival time given.

Avg. time.

$$8.2$$

|    |    |       |    |    |    |
|----|----|-------|----|----|----|
| P2 | P5 | P1    | P3 | P4 |    |
| 0  | 1  | 1+5=6 | 16 | 18 | 19 |

$$8+10=$$

Aging time.